Java Silver（SE 17・1Z0-825-JPN）　5 継承とポリモーフィズム　#11

継承とポリモーフィズム

メソッドは「中身」で、フィールドと static は「変数の型」で決まる

各ページの下に、根拠にした Oracle の資料（JLS 5.1・8.1・8.2・8.4.8・8.8・12.5・15.11・15.12・15.16）を書いた（青い文字を押すと開く）。コードは Claude が作り、JDK 17 で動かして確かめた

全体像 試験内容チェックリスト 5（継承とポリモーフィズム）

1 継承とコンストラクタ
extends は 1 つだけ。子のコンストラクタの先頭に super(); が入る

2 オーバーライドと隠蔽
同じシグネチャで書き直す。アクセス・戻り値・throws の決まり

3 ポリモーフィズムとキャスト
呼べるかは変数の型、動くのは中身。ダウンキャストは実行時に検査

4 abstract と final
抽象クラスは new できない。final は「これ以上変えさせない」

何で決まる？	決まる時期
インスタンスメソッドの**中身**	**実行時（オブジェクトのクラス）**
呼べるメソッド・使えるフィールド	**コンパイル時（変数の型）**
フィールドの値（同じ名前があるとき）	**コンパイル時（変数の型）**
static メソッド	**コンパイル時（変数の型）**
オーバーロードの選択	**コンパイル時（引数の型）**

この表がこの分野の芯。「実行時に中身で決まる」のはオーバーライドされたインスタンスメソッドだけ。

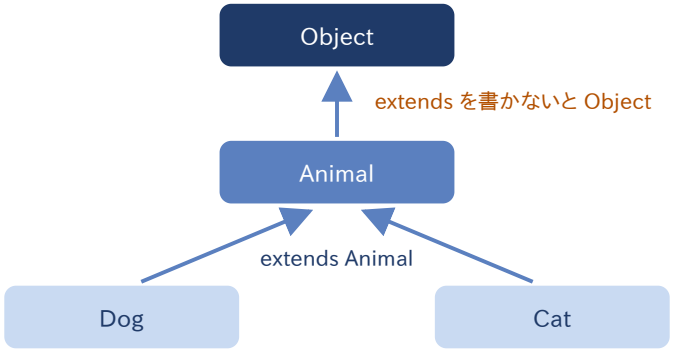
出典：Oracle「Java SE 17 Programmer I（1Z0-825-JPN）」試験内容チェックリスト／JLS 15.11.1・15.12.4.4／結果は JDK 17 で確かめたもの

第1章

継承とコンストラクタ

親のメンバを引き継ぎ、親のコンストラクタから順に動く。

継承の形と、引き継がれるもの




```
class Dog extends Animal { }        // OK
class B extends A, Object { }     // '{' expected
```

親のもの	子に引き継がれるか
public・protected のメンバ	**引き継がれる**
修飾子なしのメンバ	**同じパッケージの子だけ**
private のメンバ	**引き継がれない（n has private access in A）**
コンストラクタ・初期化子	**メンバではないので引き継がれない**

> **単一継承：**extends に書けるのは 1 クラスだけ。A → B → C と何段も重ねるのは OK。インタフェースは複数 implements できる（#12）。

private のフィールドも子のオブジェクトの中には存在する。名前で直接使えないだけで、親の public メソッドを通せば読める。

出典：JLS 8.1.4（The class Object has no direct superclass type. For a class other than Object with a normal class declaration, the direct superclass type is Object）・8.2（Members of a class that are declared private are not inherited by subclasses of that class）／結果は JDK 17 で確かめたもの

コンストラクタは親から順に動く

3 段の継承（Claude が作成）

```
class A { A() { System.out.print("A "); } }
class B extends A { B() { System.out.print("B "); } }
class C extends B { C() { System.out.print("C "); } }
// main：new C();
```

```
A B C
```

C()
super(); ← 自動

B()
super(); ← 自動

A()
super(); → Object

呼び出しは C → B → A と上へ

本体の実行は A → B → C と下へ戻りながら

子のコンストラクタの先頭	コンパイラの扱い
`super(…);` を書いた	**そのまま**
`this(…);` を書いた	**そのまま（呼んだ先が super を呼ぶ）**
どちらも書いていない	**`super();` を自動で入れる**
コンストラクタを 1 つも書いていない	**デフォルトコンストラクタ `B() { super(); }` を作る**

抽象クラスのコンストラクタも動く。new できないだけで、サブクラスを new すると super(); から呼ばれる。

出典：JLS 8.8.7（the constructor body implicitly begins with a superclass constructor invocation "super();", an invocation of the constructor of its direct superclass that takes no arguments）・8.8.9（If a class contains no constructor declarations, then a default constructor is implicitly declared）／結果は JDK 17 で確かめたもの

super(…) と this(…) の決まり

this(…) を通る流れ（Claude が作成）

```
class A { A(int x) { System.out.print("A" + x + " "); } }
class B extends A {
  B() { super(5); System.out.print("B "); }
  B(int y) { this(); System.out.print("B" + y + " "); }
}
// main：new B(9);
```

```
A5 B B9
```

B(9) → this() で B() → super(5) で A(5)。本体は A(5) → B() の残り → B(int) の残り、の順。

書き方（A には A(int) だけ）	結果（JDK 17）
`B() { print("B "); }`	constructor A in class A cannot be applied to given types;
`class B extends A { }`	同じエラー（class B の行）
`B() { print("B "); super(1); }`	call to super must be first statement in constructor
`B() { super(1); print("B "); }`	OK

親に引数なしのコンストラクタがないときは、子のどのコンストラクタも super(値) か this(…) で始める必要がある。

出典：JLS 8.8.7（The first statement of a constructor body may be an explicit invocation of another constructor of the same class or of the direct superclass）／結果は JDK 17 で確かめたもの

第2章

オーバーライドと隠蔽

同じシグネチャで書き直す。インスタンスならオーバーライド、static なら隠蔽。

オーバーライドのチェックリスト

項目	決まり	OK の例（親 → 子）	NG の例と JDK 17 のメッセージ
シグネチャ	名前と引数の型の並びが同じ	`m(int)` → `m(int)`	`m(long)` はオーバーロード。@Override を付けると method does not override or implement a method from a supertype
戻り値	同じ型。参照型ならサブクラス型も可（共変戻り値）	`A get()` → `B get()`	`int` → `long`：return type long is not compatible with int
アクセス	同じか広く	なし → `public`	`public` → なし：attempting to assign weaker access privileges; was public
検査例外	増やせない（減らす・なくすは可）	`throws Exception` → なし	なし → `throws Exception`：cannot override
static	インスタンスと static は混ぜられない	static → static（隠蔽）	overriding method is static ／ overridden method is static
final	final メソッドは書き直せない	—	overridden method is final

> NG の多くは 1 行目が同じ「m() in B cannot override m() in A」。理由は 2 行目に出る。試験では理由ごとに見分ける。

出典：JLS 8.4.8.3（The access modifier of an overriding or hiding method must provide at least as much access as the overridden or hidden method／may not be declared to throw more checked exceptions than the overridden or hidden method）・8.4.8.1（It is a compile-time error if the overridden method, mA, is a static method）・8.4.3.3／結果は JDK 17 で確かめたもの

オーバーライドとオーバーロードを見分ける

引数の型がちがうと別のメソッド（Claude が作成）

```
class A { void m(int x)  { print("A.int "); } }
class B extends A { void m(long x) { print("B.long "); } }

A a = new B(); a.m(1);    // A.int（A 型には m(int) だけ）
B b = new B(); b.m(1);    // A.int（int にぴったり）
b.m(1L);                  // B.long
```

```
A.int A.int B.long
```

print は System.out.print の略として書いた。

	オーバーライド	オーバーロード
引数	同じ	型か数がちがう
どこに書く	サブクラス	同じクラスでも子でも
選ばれる時期	実行時（中身のクラス）	コンパイル時（引数の型）
戻り値・アクセス	決まりあり	自由

@Override は安全装置。付けたメソッドがオーバーライドになっていなければコンパイルエラーにしてくれる。付けなくてもオーバーライドはできる。

出典：JLS 9.6.4.4 @Override（If a method declaration in class or interface Q is annotated with @Override, then one of the following three conditions must be true, or a compile-time error occurs）／JLS 8.4.8.1／結果は JDK 17 で確かめたもの

隠蔽：static メソッドとフィールドは「変数の型」

```
class A {
  static String who() { return "A"; }
  String me() { return "A"; }
  int v = 1;
}
class B extends A {
  static String who() { return "B"; }   // 隠蔽
  String me() { return "B"; }            // オーバーライド
  int v = 2;                             // フィールドの隠蔽
  int both() { return v * 10 + super.v; }
}
A a = new B();
```

式	結果	決め手
a.who()	A	static → 変数の型 A
a.me()	B	インスタンス → 中身 B
a.v	1	フィールド → 変数の型 A
((B) a).v	2	式の型が B
((B) a).both()	21	super.v は隠された A の v

フィールドはポリモーフィズムにならない。親のメソッドの中で name と書けば、親の name を指す（子に同名があっても）。

出典：JLS 15.11.1（Note that only the type of the Primary expression, not the class of the actual object referred to at run time, is used in determining which field to use）／JLS 8.4.8.2（If a class C declares or inherits a static method m, then m is said to hide any method m' …）／結果は JDK 17 で確かめたもの

super.メソッドと private メソッド

親の処理に足す（Claude が作成）

```
class A { void show() { print("A "); } }
class B extends A {
  void show() { super.show(); print("B "); }
}
// new B().show();
```

```
A B
```

super. を付けると、親のメソッドを直接呼ぶ（中身による切り替えをしない）。だから B の show() に戻って無限に呼び出すことはない。

private はオーバーライドされない（Claude が作成）

```
class A {
  private void secret() { print("A "); }
  void call() { secret(); }
}
class B extends A {
  void secret() { print("B "); }  // 別のメソッド
}
// new B().call(); new B().secret();
```

```
A B
```

A の call() の中の secret() は A の private メソッドに決まっている。中身が B でも切り替わらない。

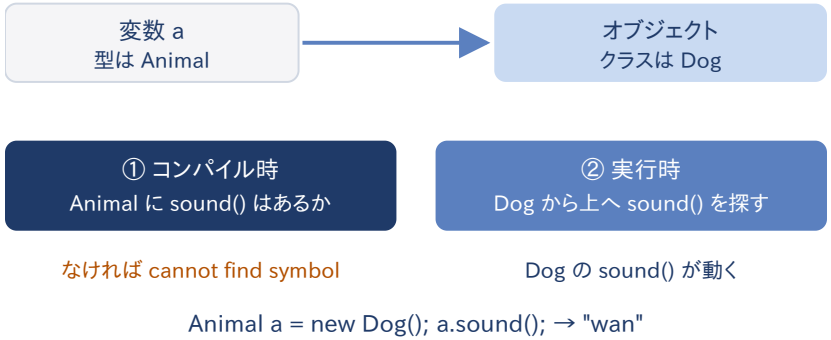
出典：JLS 15.12.4.4（If the invocation mode is super, overriding is not allowed. Method m of class or interface C is the one to be invoked）／JLS 8.4.3.3（A private method and all methods declared immediately within a final class (§8.1.1.2) behave as if they are final）／結果は JDK 17 で確かめたもの

第3章

ポリモーフィズムとキャスト

同じ呼び出しで、中身ごとにちがう処理。型を戻すときは検査がいる。

メソッド呼び出しは 2 段階で決まる



中からの呼び出しも切り替わる（Claude が作成）

```
class A {
  void m() { print("A.m "); n(); }  // this.n()
  void n() { print("A.n "); }
}
class B extends A { void n() { print("B.n "); } }
// A a = new B(); a.m();
```

```
A.m B.n
```

Animal a = new Dog(); のあと	結果
a.sound()（Dog がオーバーライド）	Dog の sound()
a.bark()（Dog だけにある）	cannot find symbol
((Dog) a).bark()	OK

出典：JLS 15.12.1（the type to search is the declared type T of the variable denoted by ExpressionName）・15.12.4.4（The lookup procedure starts from class R, the actual run-time class of the target object）／結果は JDK 17 で確かめたもの

キャストの 3 つの場合

コード	結果（JDK 17）
`Animal a = new Dog();`	OK（アップキャスト、キャスト不要）
`Dog d = a;`	incompatible types: Animal cannot be converted to Dog
`Dog d = (Dog) a;`	OK（中身が Dog）
`Cat c = (Cat) a;`	実行時 ClassCastException（class Dog cannot be cast to class Cat）
`Cat c = (Cat) dog;`（dog は Dog 型）	incompatible types: Dog cannot be converted to Cat

> **コンパイラは変数の型だけを見る。**「中身が合うかもしれない」なら通し、実行時に検査する。「ありえない」なら止める。

アップキャスト（子 → 親）
いつも安全。書かなくてよい。例外なし

ダウンキャスト（親 → 子）
(Dog) が必要。中身が合わなければ ClassCastException

継承関係のないクラスどうし
(Cat) を書いてもコンパイルエラー

安全にするには、先に instanceof で調べる。Java 16 からは `if (a instanceof Dog d)` で調べるとキャストを一度に書ける（#10）。

出典：JLS 5.1.5（Widening reference conversions never require a special action at run time and therefore never throw an exception at run time）・5.1.6.3（If T is a class type, then R must be either the same class as T (§4.3.4) or a subclass of T, or a ClassCastException is thrown）／JLS 15.16／結果は JDK 17 で確かめたもの

落とし穴：親のコンストラクタから子のメソッド

```
class A {
  A() { show(); }
  void show() { System.out.println("A.show"); }
}
class B extends A {
  int x = 5;
  void show() { System.out.println("B.show " + x); }
}
// main：new B();
```

```
B.show 0
```

順	new B() の中で起きること	x
1	B のフィールドを既定値にする	0
2	B() の先頭の super(); で A() へ	0
3	A() の show() → 中身は B なので B の show()	0 を表示
4	A() が終わり、B のフィールド初期化 x = 5	5
5	B() の残りの本体	5

作っている途中でもオーバーライドは効く。子のフィールドはまだ初期化前なので、int は 0、参照は null が見える。

出典：JLS 12.5（If methods are invoked that are overridden in subclasses in the object being initialized, then these overriding methods are used, even before the new object is completely initialized）／結果は JDK 17 で確かめたもの

第4章

abstract と final

abstract は「子に書かせる」、final は「これ以上変えさせない」。

抽象クラスと抽象メソッド

```
abstract class Shape {
  abstract double area();                // 中身なし
  String describe() { return "area=" + area(); }
}
class Sq extends Shape {
  double s;
  Sq(double s) { this.s = s; }
  double area() { return s * s; }   // 実装
}
Shape sh = new Sq(3);                    // 変数の型には使える
System.out.println(sh.describe());
```

```
area=9.0
```

書き方	結果（JDK 17）
`new Shape()`	Shape is abstract; cannot be instantiated
`class Sq extends Shape { }`	Sq is not abstract and does not override abstract method area() in Shape
`abstract class Sq2 extends Shape { }`	OK（実装を先送り）
abstract のないクラスに抽象メソッド	Shape is not abstract and does not override …
`abstract double area() { return 0; }`	abstract methods cannot have a body

抽象クラスはコンストラクタ・フィールド・中身のあるメソッドも持てる。抽象メソッドが 0 個でも abstract にできる。

出典：JLS 8.1.1.1（It is a compile-time error if an attempt is made to create an instance of an abstract class using a class instance creation expression）・8.4.3.1（Every subclass of A that is not abstract (§8.1.1.1) must provide an implementation for m, or a compile-time error occurs）／結果は JDK 17 で確かめたもの

final の 3 つの使い方

付ける所	意味	破ったときの JDK 17 のメッセージ
クラス	継承できない	cannot inherit from final A
メソッド	オーバーライド・隠蔽できない	m() in B cannot override m() in A ／ overridden method is final
変数	値を 1 回しか入れられない（#03）	cannot assign a value to final variable

> **final メソッドを持つクラスは継承できる。**継承を止めるのは final クラスだけ。String は final クラス。

組み合わせ	JDK 17
`final abstract class A`	illegal combination of modifiers: abstract and final
`private abstract void m();`	illegal combination of modifiers: abstract and private
`static abstract void m();`	illegal combination of modifiers: abstract and static
final クラスの中のメソッド	自動で final と同じ扱い

abstract は「子で完成させる」、final は「完成していて変えさせない」。逆の意味なので一緒に付けられない。

出典：JLS 8.1.1.2（It is a compile-time error if a class is declared both final and abstract, because the implementation of such a class could never be completed）・8.1.4（It is a compile-time error if the ClassType names a class that is final）・8.4.3.3（A method can be declared final to prevent subclasses from overriding or hiding it）／結果は JDK 17 で確かめたもの

問われ方とまちがえやすい点

こう聞かれたら	答えの考え方
コンストラクタの表示順	親から順。this(…) を通るときは呼ばれた側の本体が先
子のコンストラクタがコンパイルできるか	親に引数なしのコンストラクタがあるか。super(…) は先頭か
オーバーライドがコンパイルできるか	アクセスを狭めていないか・戻り値・検査例外・static と final
親の型の変数での表示	インスタンスメソッドは中身、フィールドと static は変数の型
キャストの結果	キャストなしの代入はエラー。継承関係なしはエラー。中身が合わなければ ClassCastException
親のコンストラクタから呼ぶメソッド	子のメソッドが動き、子のフィールドはまだ 0 や null
abstract・final の組み合わせ	抽象クラスは new 不可、変数の型は OK。final と abstract は一緒に付けない

出典：このPDFの各ページの資料

まとめ

継承とコンストラクタ
extends は 1 つ。super(); が自動で先頭に。親から順に動く

オーバーライド
同じシグネチャ・アクセスは同じか広く・共変戻り値・例外は増やさない

ポリモーフィズム
呼べるかは変数の型、動くのは中身。フィールド・static は変数の型

abstract と final
抽象クラスは new 不可。final はクラス・メソッド・変数

こんなとき	結果
public を修飾子なしでオーバーライド	コンパイルエラー
int を返すメソッドを long でオーバーライド	コンパイルエラー
親の型の変数で static メソッド	親のメソッド
中身が Dog の Animal を (Cat)	ClassCastException
new 抽象クラス	コンパイルエラー

次は #12 インタフェースとシール・クラス。implements・default メソッド・sealed と permits へ進む。

出典：このPDFの各ページの資料